SIMATS ENGINEERING
ASSESSMENT TOOL 3
Mock Interview

COURSE NAME: COMPUTER VISION WITH OPENCV
COURSE CODE: DSA0204

COURSE OUTCOME COVERED

CO5: Understand video processing - motion computation - 3D vision - geometry. (BTL 6)

Assessment Tool Weightage: 10%

Code of Conduct:

I ___ K Sampath Reddy (192424285) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *K Sampath Reddy*

Mock Interview Questions

OpenCV Basics & Pipeline Thinking

1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.
2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.
3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.
4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

Image & Video Processing

5. A video stream drops frames during processing. Analyze the problem and redesign your approach to ensure smooth playback, explaining trade-offs and optimization techniques.
6. You need to process only every 5th frame of a video. Design an efficient solution and justify your approach, considering performance and accuracy.
7. A system must process both images and videos using the same pipeline. Propose a generalized design and explain how it handles both cases effectively.

8. You need to extract frames from a video and save only those with significant changes. Analyze the problem and design a method, explaining your decision criteria and approach.

Feature Detection & Drawing

9. You are asked to highlight key features in an image. Design a feature detection and visualization approach, explaining the techniques and their suitability.
10. A system must draw bounding boxes and labels for detected objects. Explain how you would structure this module, including reasoning behind chosen methods.
11. You need to compare two images and highlight differences visually. Analyze the problem and design a solution, justifying your approach.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

SOLUTIONS

OpenCV Basics & Pipeline Thinking

Q1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.

List image paths with a directory walk filtered by valid extensions, then loop through them one at a time: read, validate (empty/corrupt check), process, display/save, and release before the next file. Keeping discovery, I/O, and processing in separate functions makes the pipeline reusable (the same processing function later works on video frames too). Streaming one image at a time avoids loading the whole folder into memory. Challenges: corrupted files (handled with a log-and-continue guard) and large folders (addressed with multithreaded I/O since reading/decoding is I/O-bound, while heavier processing stays sequential or GPU-accelerated).

Q2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.

Check cap.isOpened() first to confirm the capture object opened at all. If not, verify the file path resolves correctly (most common cause), then check codec/container support — OpenCV depends on the system FFmpeg/GStreamer backend, so an unusual codec can silently fail. Run cv2.getBuildInformation() to confirm the installed OpenCV build actually has video backend support. If it opens but reads fail early, the file may be truncated/corrupted — test in another player and compare frame count to expected duration. As a fallback, re-encode with FFmpeg to H.264/MP4 and retest; success confirms the root cause was format compatibility.

Q3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.

Inside the capture loop, draw directly on each frame before display using cv2.putText for labels/timestamps and cv2.rectangle/circle/line for shapes, computing coordinates once per frame rather than recalculating unnecessarily. For efficiency, pre-compute static overlay elements (e.g., a fixed legend) once outside the loop, and use simple primitive draws rather than complex compositing, since these OpenCV drawing functions are already highly optimized in C++. If overlays involve semi-transparency, use cv2.addWeighted on a separate overlay layer rather than per-pixel blending in Python, which would be far slower.

Q4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

First rule out a read-path issue (wrong color space assumption, or the image genuinely being underexposed) by inspecting the raw pixel histogram. If it's genuinely underexposed, apply gamma correction or histogram equalization (CLAHE for uneven lighting, since it operates locally and avoids over-brightening already-bright regions) to redistribute intensity values toward the mid-range. I'd apply this on the luminance channel only (e.g., convert to LAB/YCrCb, equalize L/Y, convert back) rather than each RGB channel independently, which preserves color balance while genuinely improving visibility instead of introducing color casts.

Image & Video Processing

Q5. A video stream drops frames during processing. Analyze the problem and redesign your approach to ensure smooth playback, explaining trade-offs and optimization techniques.

Profile to find the actual bottleneck (capture, processing, or display) rather than optimizing blindly. Decouple capture from processing using a producer-consumer pattern with threads and a bounded queue, so the camera driver never blocks. If processing genuinely can't keep pace, trade completeness for smoothness via frame skipping, lower processing resolution, or a lighter model. Trade-off: threading and skipping improve throughput but reduce temporal completeness — acceptable for live monitoring, but for tasks needing every frame, invest in faster models/hardware instead of skipping data.

Q6. You need to process only every 5th frame of a video. Design an efficient solution and justify your approach, considering performance and accuracy.

Maintain a frame counter, read every frame sequentially (safest across codecs), and only process when counter % 5 == 0, discarding the rest immediately. For formats with reliable seeking, cap.set(CAP_PROP_POS_FRAMES) can skip decode cost for skipped frames, saving more compute but risking frame-accuracy issues with B-frames. On accuracy: sampling every 5th frame (~6 samples/sec at 30fps) is fine for general monitoring but risky for fast events, so the interval should be a tunable parameter validated against expected motion speed, not a fixed constant.

Q7. A system must process both images and videos using the same pipeline. Propose a generalized design and explain how it handles both cases effectively.

Treat a single image as a video with exactly one frame: build a common FrameSource abstraction with a next_frame() method, backed by either cv2.VideoCapture (video/webcam) or a one-shot wrapper around cv2.imread (image). The processing function operates on whatever frame it

receives, unaware of the source type. This generalization means all downstream logic (preprocessing, detection, drawing) is written once and works identically for both cases, and adding a new source type (e.g., an image folder) later only requires a new FrameSource implementation, not pipeline changes.

Q8. You need to extract frames from a video and save only those with significant changes. Analyze the problem and design a method, explaining your decision criteria and approach.

Read sequentially, and for each candidate frame compute a lightweight similarity measure against the last-saved frame — either mean absolute pixel difference on a downscaled grayscale copy, or histogram correlation, both cheap to compute. If the difference exceeds a tuned threshold, save the frame and update the reference; otherwise discard it. Downscaling before comparison keeps the check fast without needing full-resolution accuracy. The threshold is the key design decision: too low saves near-duplicates, too high misses genuine changes, so I'd tune it against sample footage from the actual deployment scenario.

Feature Detection & Drawing

Q9. You are asked to highlight key features in an image. Design a feature detection and visualization approach, explaining the techniques and their suitability.

Clarify what 'features' means: ORB for general-purpose distinctive keypoints (fast, license-free, rotation-invariant — good default for matching/tracking), Harris/Shi-Tomasi for structural corners (cheap, well-localized), or Canny for edges/shape outlines. Detect on a grayscale copy but draw results (cv2.drawKeypoints with rich circles showing size/orientation) back onto the original color

image for clearer visualization. Trade-off: ORB/Shi-Tomasi are fast enough for real-time use but less distinctive on low-texture images, while SIFT is more robust to scale/illumination but heavier — choice depends on whether this runs once on a static image or continuously on video.

Q10. A system must draw bounding boxes and labels for detected objects. Explain how you would structure this module, including reasoning behind chosen methods.

Keep detection and drawing as separate functions: detection returns a structured list of (box, label, confidence) tuples, and a dedicated draw_detections() function consumes that list purely for rendering, never recomputing anything. This separation lets the drawing style (colors, font, box thickness) be changed without touching detection logic. I'd color-code boxes by class for quick visual scanning, draw the label on a small filled background rectangle above the box for legibility against busy backgrounds, and clip label positions so they don't run off-frame at image edges.

Q11. You need to compare two images and highlight differences visually. Analyze the problem and design a solution, justifying your approach.

Align sizes first (resize the second image to match the first if needed), convert both to grayscale, blur lightly to suppress noise, then compute absolute pixel difference and threshold it to a binary mask. Clean the mask with morphological opening/closing to remove residual noise speckles, then find contours and draw bounding boxes on both images only where contour area exceeds a minimum threshold. Blurring before differencing and area-filtering after thresholding are the two steps that specifically prevent sensor noise from being misreported as a 'difference'.